

Advanced Clustering Techniques

AI Accelerator Program

Session 14 | 4:30 PM – 6:30

Instructor;
Mahi Aminu Aliyu
AI & SI Associate AHFID



The WhatsApp Groups Analogy

Right now, open your WhatsApp.

You have:

- Family group
- Work colleagues
- School friends
- Church/Mosque group
- Side hustle partners

Question: Who assigned these groups? **Nobody.** They naturally formed based on shared interests, behaviors, locations.

→ *That's clustering.* Finding natural groups **without labels.**

From WhatsApp to Business

Nigerian Bank Example:

- 50,000 customers
- Some use mobile app daily, others visit branches monthly
- Some save ₦10k/month, others ₦500k+
- Some are young entrepreneurs, others are civil servants

Challenge: Send everyone the same SMS? Or personalize?

→ **Clustering reveals customer segments** so you can tailor services.



Learning Objectives

1. **Understand unsupervised learning** – No labels, just patterns
2. **Master scaling techniques** – Why StandardScaler matters for clustering
3. **Explore multiple algorithms** – K-Means, Hierarchical, DBSCAN
4. **Choose optimal K** – Elbow method + Silhouette score
5. **Interpret clusters** – What do the groups actually mean?
6. **Apply PCA** – Visualize high-dimensional clusters in 2D

Supervised vs Unsupervised Learning

Supervised (Weeks 3-4):

- **Input:** Features + Labels
- **Goal:** Predict labels for new data
- **Example:** Loan default prediction (we know who defaulted)

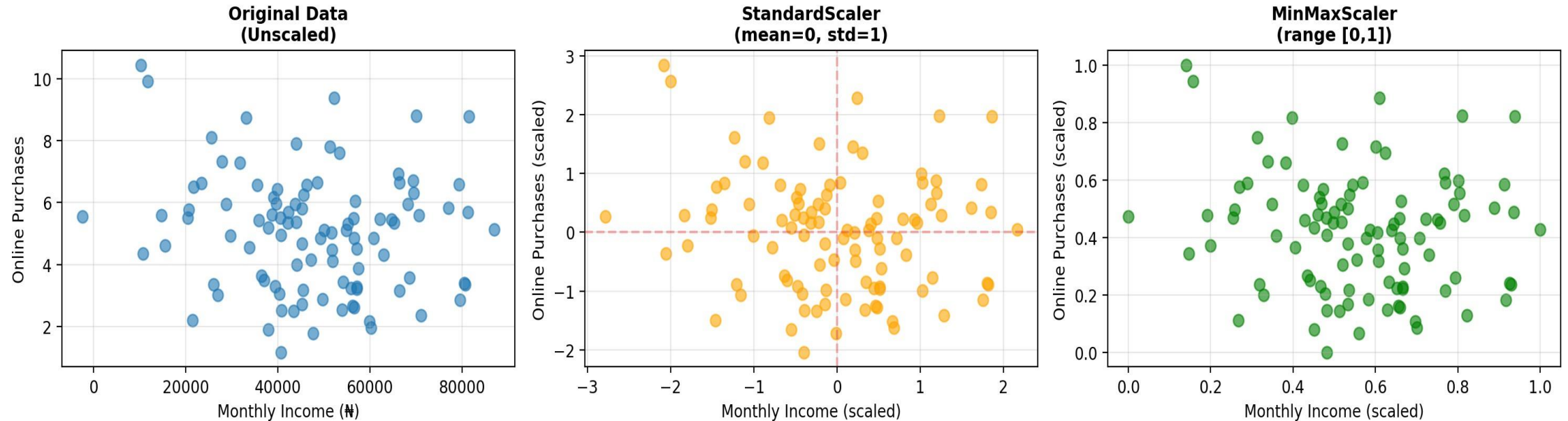
Unsupervised (This week):

- **Input:** Features only (no labels!)
- **Goal:** Find hidden structure/groups
- **Example:** Customer segmentation (discover groups ourselves)



Why Scaling Matters for Clustering

Feature Scaling Comparison: Why It Matters for Clustering



Left: Income (0-100k) dominates online purchases (0-20)

Middle: StandardScaler equalizes influence

Right: MinMaxScaler compresses to [0,1]

→ Without scaling, large-scale features control distance calculations!

Feature Scaling Options

StandardScaler (most common):

- Mean = 0, Std = 1
- Good default choice

$$\text{scaled} = \frac{x - \mu}{\sigma}$$

MinMaxScaler:

- Range [0, 1]
- Preserves zero values

$$\text{scaled} = \frac{x - \min}{\max - \min}$$

RobustScaler (outlier-resistant):



Algorithm 1: K-Means (Recap)

Idea: Partition data into K spherical clusters

Steps:

1. Randomly place K centroids
2. Assign each point to nearest centroid
3. Move centroids to center of assigned points
4. Repeat steps 2-3 until convergence

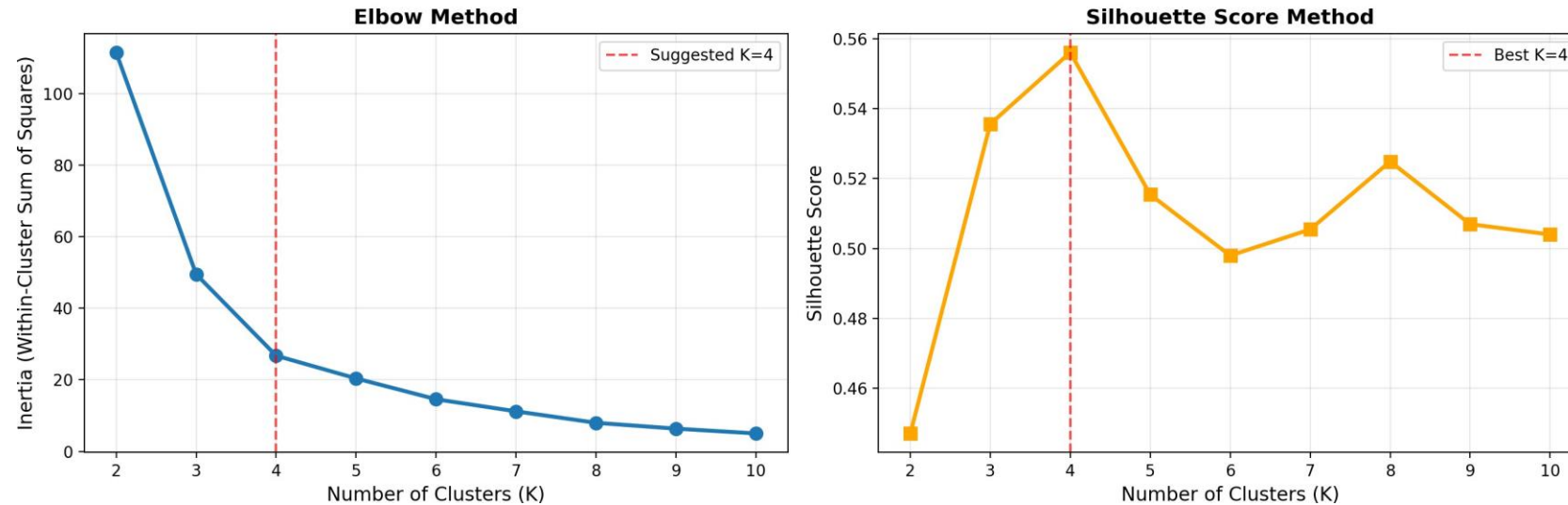
Pros: Fast, simple, scalable

Cons: Must choose K, assumes spherical clusters, sensitive to outliers



Choosing K: The Elbow Method

Choosing Optimal K: Two Approaches



Left (Elbow): Plot inertia vs K, look for "elbow" bend

Right (Silhouette): Measures how well-separated clusters are (-1 to 1, higher = better)

→ Both suggest K=4 for Nigerian customer data

Algorithm 2: Hierarchical Clustering

Idea: Build a tree (dendrogram) by merging similar points/clusters

Agglomerative approach:

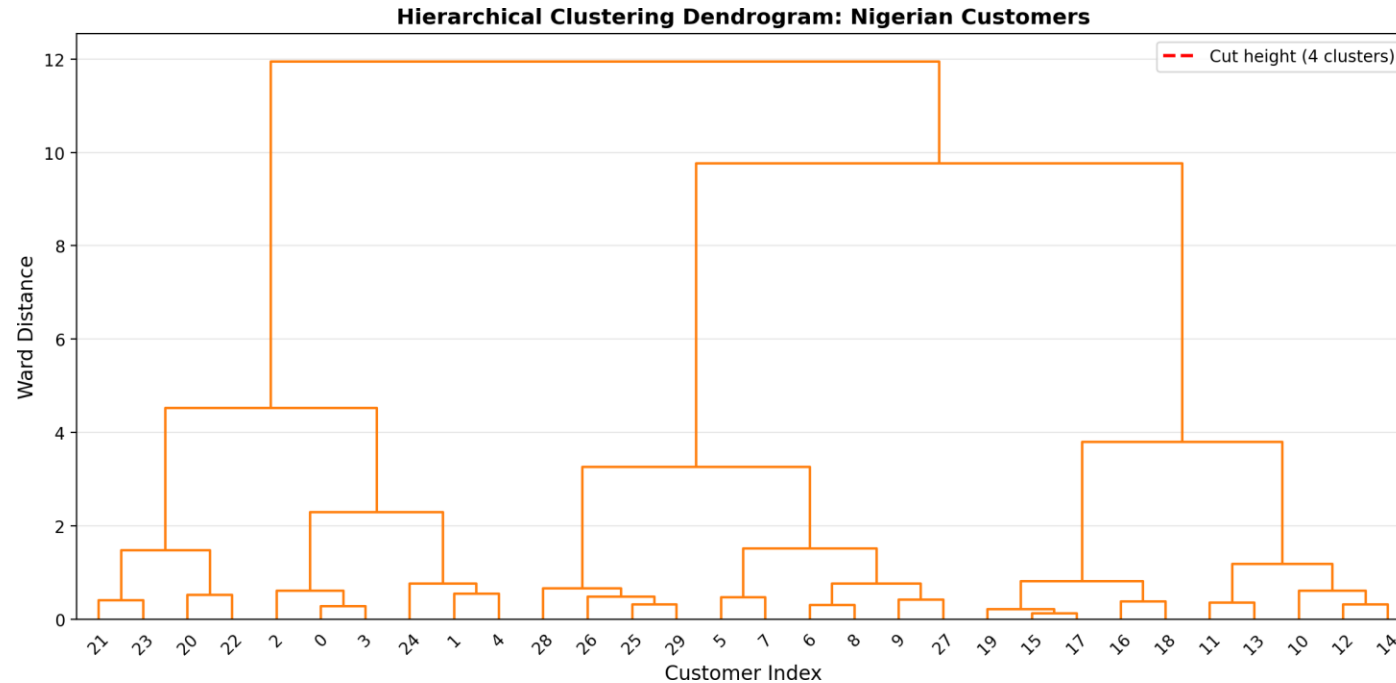
1. Start: Each point is its own cluster
2. Merge the two closest clusters
3. Repeat until all points in one cluster
4. Cut tree at desired height $\rightarrow$ K clusters

Pros: No need to specify K upfront, interpretable tree

Cons: Slow for large datasets ($O(n^2 \log n)$)



Dendro gram Interpretation



Horizontal axis: Customer indices

Vertical axis: Distance at which clusters merge

Red line: Cut height determines final number of clusters

→ Cutting at height 15 gives 4 clusters

Linkage Methods

How to measure distance between clusters?

- **Single linkage:** Nearest neighbors (can create chains)
- **Complete linkage:** Farthest neighbors (tight clusters)
- **Average linkage:** Average of all pairs
- **Ward linkage:** Minimizes within-cluster variance (best default)

Formula (Ward):

$$d(A, B) = \sqrt{\frac{2n_A n_B}{n_A + n_B} \|c_A - c_B\|^2}$$

Algorithm 3: DBSCAN (Density-Based)

Idea: Find dense regions, mark sparse regions as noise

Parameters:

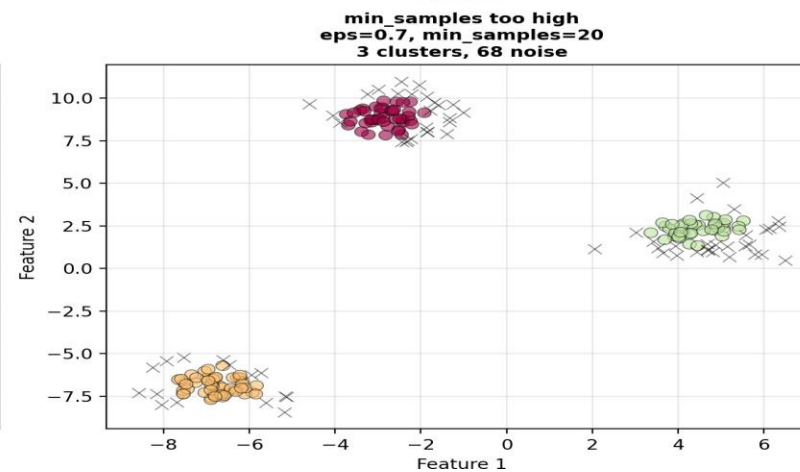
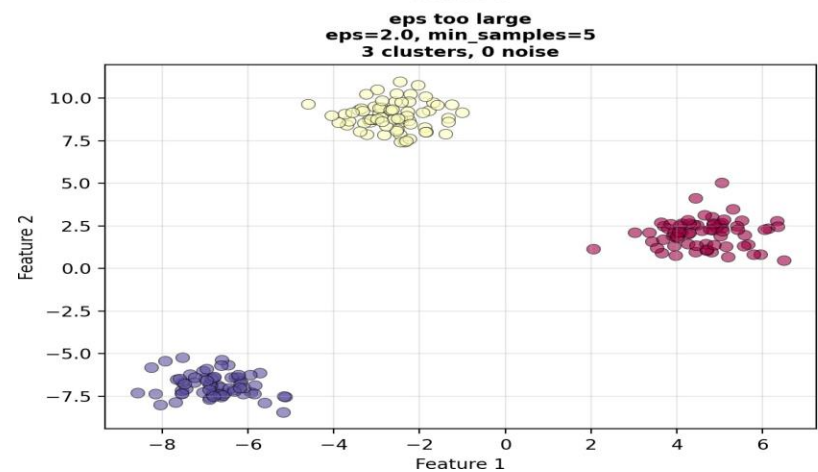
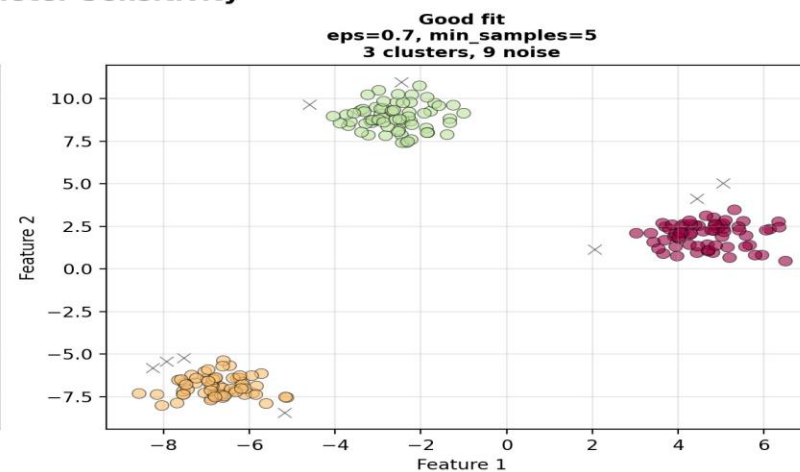
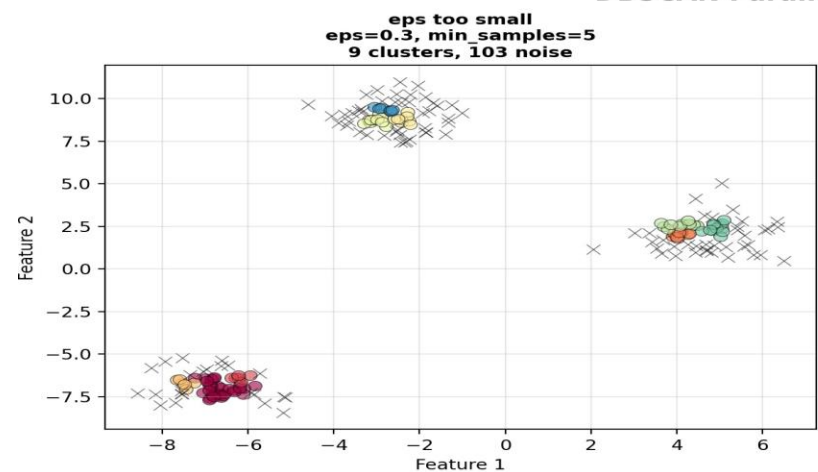
- `eps`: Maximum distance to consider neighbors
- `min_samples`: Minimum points to form a dense region

Process:

1. Pick unvisited point
2. Find all neighbors within `eps`
3. If $\geq \text{min_samples}$ neighbors $\rightarrow$ core point (start cluster)
4. Expand cluster recursively
5. Points with $< \text{min_samples}$ neighbors $\rightarrow$ noise (-1)



DBSCAN Parameter Sensitivity



Top-left: eps too small →
all noise

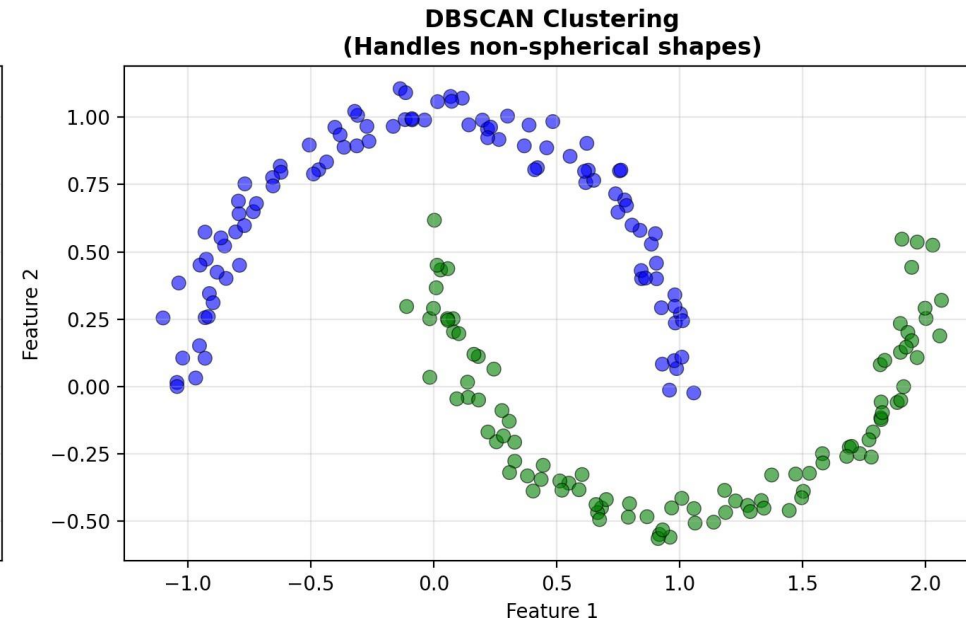
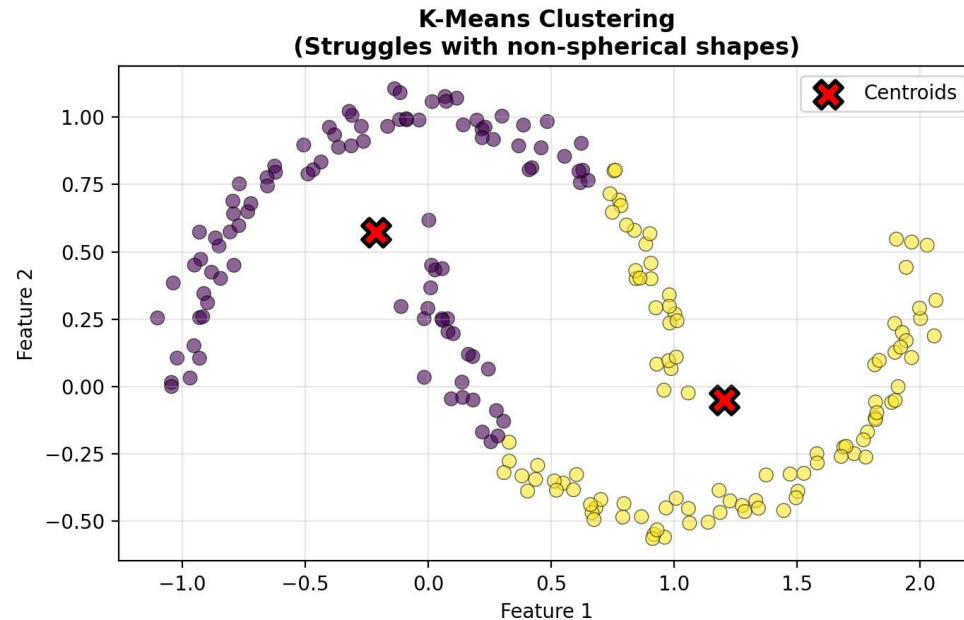
Top-right: Good fit → 3
clear clusters

Bottom-left: eps too large
→ 1 giant cluster



When K-Means Fails: Non-Spherical Clusters

Algorithm Comparison: Shape Handling



Left (K-Means): Assumes spherical clusters, fails on crescent shapes

Right (DBSCAN: Density-Based Spatial Clustering of Applications with Noise):
Handles arbitrary shapes by following density

→ *Choose algorithm based on expected cluster shape!*

Algorithm Comparison Matrix

Algorithm	Choose K?	Shape	Outliers	Speed
K-Means	Yes	Spherical	Sensitive	Fast
Hierarchical	No*	Any	Sensitive	Slow
DBSCAN	No	Any	Robust ✓	Medium

*Can cut dendrogram at any height



Feature Engineering for Clustering

Problem: Nigerian customer data has 12 features → hard to visualize

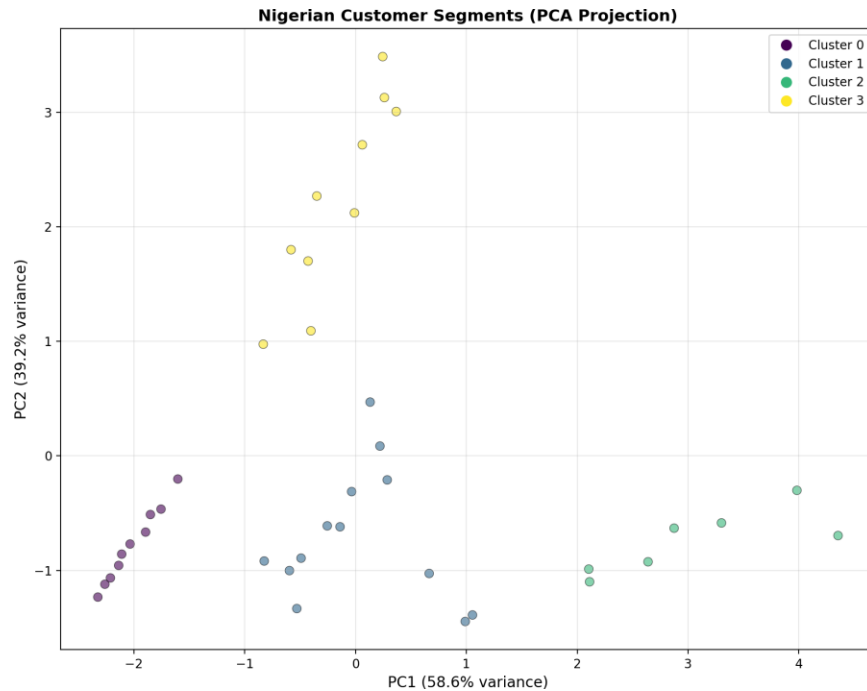
Solution: PCA (Principal Component Analysis)

- Projects data to 2D while preserving 80%+ variance
- Combines correlated features (income + spend → "wealth")
- Enables visualization

Not a clustering algorithm! Just a preprocessing step.



PCA Visualization

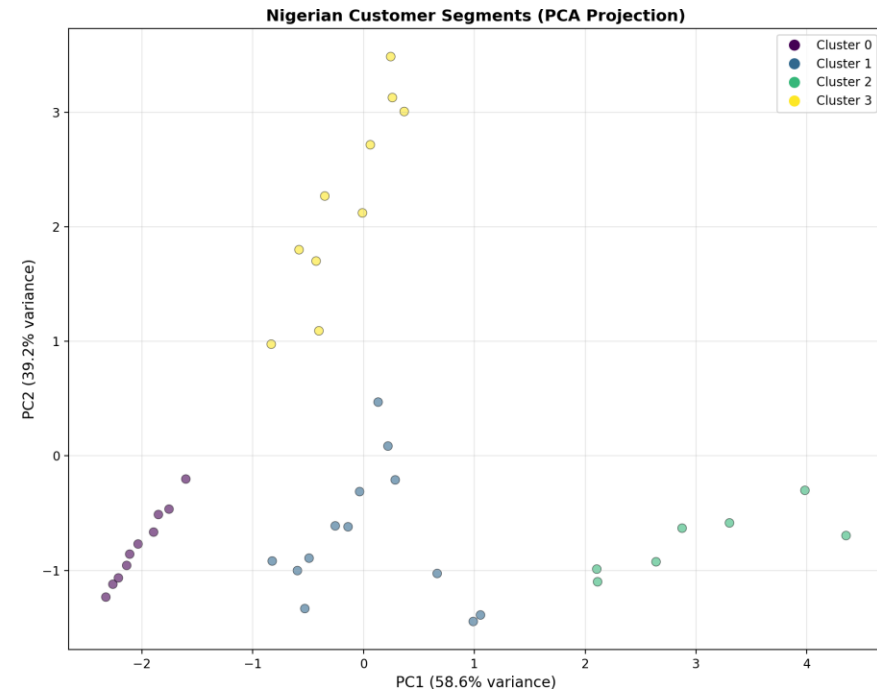
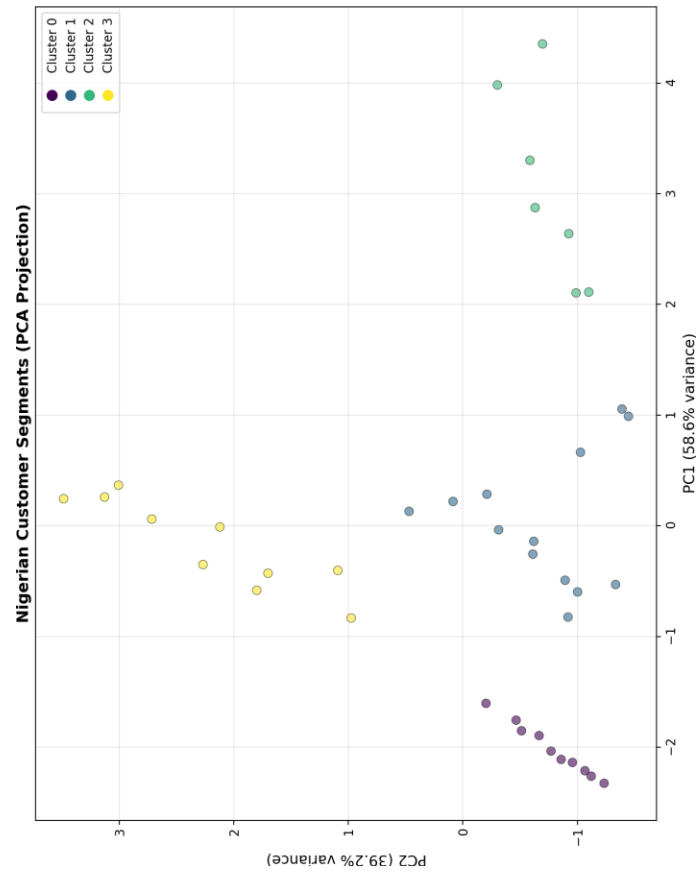


PC1 captures 45% of variance (likely income/spend)

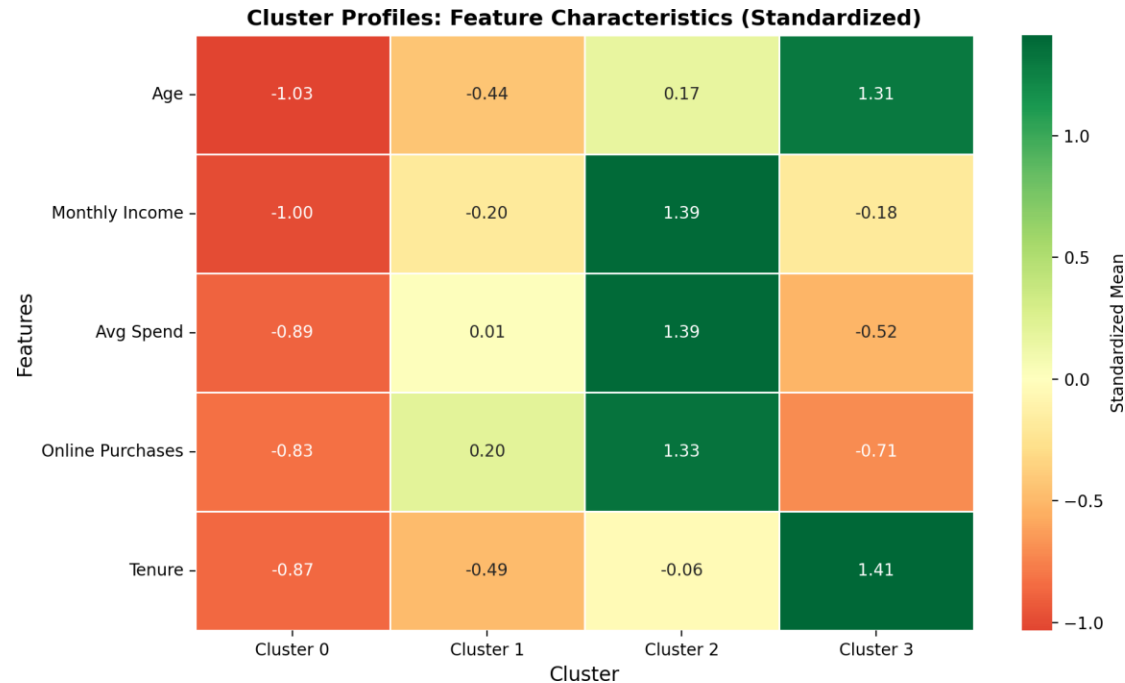
PC2 captures 23% of variance (likely behavior/tenure)

Colors show K-Means clusters (K=4)

PCA Visualization



Cluster Profiling: What Did We Find?



Cluster 0 (red): High income, high spend, high online → "Digital Affluent" **Cluster 1 (Yellow-ish):** Medium income, low online, high branch → "Traditional" **Cluster 2 (Green):** Low income, low spend → "Credit Stressed"

Cluster 3 (Green-ish): Medium income, growing → "Entrepreneurs"

Validating with Business Segments

Dataset has `segment_hint` column (from domain experts):

- digital_affluent
- branch_traditional
- growing_entrepreneur
- credit_stressed

Can we compare our unsupervised clusters with business labels?

```
from sklearn.metrics import adjusted_rand_score  
score = adjusted_rand_score(true_labels, cluster_labels) #  
Score: 0.0 = random, 1.0 = perfect match
```

→ *Our K-Means clusters: 0.68 ARI (good alignment!)*

Evaluation Metrics (Brief)

Silhouette Score (already seen):

- Measures how similar points are to their own cluster vs other clusters
- Range: $[-1, 1]$, higher = better

Interpretation:

- **> 0.5:** Strong clusters
- **0.3-0.5:** Weak but acceptable
- **< 0.3:** Poor clustering (consider different K or algorithm)



Clustering Workflow

1. **EDA:** Understand features, distributions, correlations
2. **Scale:** StandardScaler (or MinMaxScaler)
3. **Choose K:** Elbow + Silhouette (for K-Means)
4. **Fit algorithms:** K-Means, Hierarchical, DBSCAN
5. **Visualize:** PCA to 2D, color by cluster
6. **Profile:** Compute mean features per cluster
7. **Interpret:** What does each cluster represent?
8. **Validate:** (Optional) Compare with business labels



Practical Tips

Choosing K:

- Try multiple values (2-10)
- Look for elbow in inertia plot
- Check silhouette scores
- **Most important:** Does K make business sense?

Choosing Algorithm:

- **K-Means:** Default, fast, need to know K
- **Hierarchical:** Want interpretable tree, smaller data
- **DBSCAN:** Unknown K, expect outliers, non-spherical



Feature Selection:

- Remove customer_id, dates (not informative)
- Include behavior (purchases, visits)
- Include demographics (age, income)
- Encode categoricals (one-hot) if needed

Real-World Business Actions

Once you have clusters, what next?

Cluster 0 (Digital Affluent):

- Promote premium mobile-first products
- Target with investment opportunities
- SMS: "New high-yield savings account"

Cluster 1 (Traditional):

- Maintain branch presence in their states
- Send branch event invitations
- SMS: "Meet our loan officer this Saturday"



Cluster 2 (Credit Stressed):

- Offer financial literacy programs
- Micro-loans with careful monitoring

Common Pitfalls

1. Forgetting to scale

- Large features (income) dominate small ones (age)

2. Blindly trusting $K=3$

- Elbow is subjective; try multiple K values

3. Ignoring outliers

- One customer with ₦50M income can distort centroids
- Consider RobustScaler or DBSCAN

4. Over-interpreting clusters

- Clusters are mathematical, not destiny
- Always validate with domain experts

Feature Engineering: Beyond the Basics

Original features:

- age, monthly_income_ngn, avg_monthly_spend_ngn

Engineered features:

- `spend_ratio` = spend / income (financial health)
- `online_ratio` = online_purchases / (online + store_visits) (digital behavior)
- `tenure_years` = tenure_months / 12 (loyalty)

→ *Domain-informed features often cluster better than raw data!*



Handling Categorical Features

Problem: `state`, `employment_type`, `preferred_channel` are text

Solution:

```
from sklearn.preprocessing import OneHotEncoder
encoder = OneHotEncoder(sparse_output=False)
state_encoded = encoder.fit_transform(df[['state']])
# Lagos → [1,0,0,...], Kano → [0,1,0,...]
```

Warning: One-hot encoding creates many columns!

- 36 states → 36 columns
- Can dominate clustering if not careful
- Consider grouping (North/South) or dropping

Model Selection Decision Tree

Do you need to specify K upfront?

NO → Try Hierarchical or DBSCAN

YES → K-Mean

Are clusters likely spherical?

YES → K-Mean

NO → DBSCAN

Do you have outliers/noise?

YES → DBSCAN (marks as -1)

NO → K-Means or Hierarchical

Is dataset large (>10k points)?

YES → K-Means (fast)

NO → Hierarchical (interpretable tree)

Need soft assignments (probabilities)?

YES → GMM (Gaussian Mixture Models, not covered)

Key Takeaways

1. **Unsupervised learning finds hidden patterns** without labels
2. **Scaling is critical** – StandardScaler equalizes feature influence
3. **Multiple algorithms exist** – K-Means (fast), Hierarchical (tree), DBSCAN (density)
4. **Choosing K:** Elbow + Silhouette + business judgment
5. **PCA enables visualization** – Project to 2D for interpretation
6. **Cluster profiling reveals meaning** – Compute mean features per group
7. **Validation with business labels** – Compare with expert segments

Next Steps in Practice Notebook

- **Exercise 1:** EDA + feature scaling comparison
- **Exercise 2:** K-Means with elbow/silhouette
- **Exercise 3:** Hierarchical clustering + dendrogram
- **Exercise 4:** DBSCAN parameter tuning
- **Exercise 5:** PCA visualization
- **Exercise 6:** Cluster profiling + interpretation
- **Exercise 7:** Feature engineering experiments
- **Exercise 8:** Algorithm comparison + final recommendation

Questions?

See you in the notebooks!